

Unit 3 Deepseek Methods

Java String and StringBuffer Methods Explained with Examples

Here's a comprehensive explanation of the requested methods with simple and complex examples for each:

1. String Methods

a. split(String regex) and split(String regex, int limit)

****Syntax:****

```
```java
String[] split(String regex)
String[] split(String regex, int limit)
```
```

****Simple Example:****

```
```java
String fruits = "apple,banana,orange,grape";
String[] allFruits = fruits.split(","); // Splits on all commas
// Result: ["apple", "banana", "orange", "grape"]

String[] twoFruits = fruits.split(",", 2); // Splits into max 2 parts
// Result: ["apple", "banana,orange,grape"]
```
```

****Complex Example:****

```
```java
```

```
String data = "John:Doe:35:New York;Jane:Smith:28:Chicago;Mike:Johnson:42:Los Angeles";
```

```
String[] people = data.split(";"); // Split by semicolon first
```

```
for (String person : people) {
```

```
 String[] details = person.split(":", 4); // Split by colon, max 4 parts
```

```
 System.out.println("Name: " + details[0] + " " + details[1] +
```

```
 ", Age: " + details[2] + ", City: " + details[3]);
```

```
}
```

```
```
```

```
### b. regionMatches()
```

```
**Syntax:**
```

```
```java
```

```
boolean regionMatches(int toffset, String other, int ooffset, int len)
```

```
boolean regionMatches(boolean ignoreCase, int toffset, String other, int ooffset, int len)
```

```
```
```

```
**Simple Example:**
```

```
```java
```

```
String str1 = "Hello World";
```

```
String str2 = "world";
```

```
boolean match = str1.regionMatches(6, str2, 0, 5); // false (case-sensitive)
```

```
boolean ignoreCaseMatch = str1.regionMatches(true, 6, str2, 0, 5); // true
```

```
```
```

```
**Complex Example:**
```

```
```java
```

```
String document = "The quick brown fox jumps over the lazy dog";
```

```
String searchTerm = "BROWN FOX";
```

```
// Search for term with case insensitivity and proper bounds checking
```

```
if (document.length() >= 10 && searchTerm.length() <= document.length() - 10) {
```

```
 boolean found = document.regionMatches(true, 10, searchTerm, 0, searchTerm.length());
```

```
 System.out.println("Term found: " + found); // true
```

```
}
```

```
'''
```

```
c. getChars()
```

```
Syntax:
```

```
```java
```

```
void getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)
```

```
'''
```

```
**Simple Example:**
```

```
```java
```

```
String str = "Hello";
```

```
char[] charArray = new char[5];
```

```
str.getChars(0, 5, charArray, 0);
```

```
// charArray now contains ['H', 'e', 'l', 'l', 'o']
```

```
'''
```

```
Complex Example:
```

```
```java
```

```
String longText = "This is a long text that needs partial extraction";
```

```
char[] buffer = new char[20];
```

```
// Extract "partial extraction" from the string
int start = longText.indexOf("partial");
int end = longText.length();
if (start != -1 && end - start <= buffer.length) {
    longText.getChars(start, end, buffer, 0);
    System.out.println(new String(buffer).trim()); // "partial extraction"
}
'''
```

d. replace()

****Syntax:****

```
'''java
String replace(char oldChar, char newChar)
String replace(CharSequence target, CharSequence replacement)
'''
```

****Simple Example:****

```
'''java
String str = "apple";
String newStr = str.replace('p', 'b'); // "abble"
'''
```

****Complex Example:****

```
'''java
String html = "<div class='header'>Welcome</div><p>Hello World</p>";
String plainText = html.replace("<div class='header'>", "")
                        .replace("</div>", "\n")
                        .replace("<p>", "");
```

```
        .replace("</p>", "");  
  
// Result: "Welcome\nHello World"  
'''
```

e. substring()

****Syntax:****

```
'''java  
  
String substring(int beginIndex)  
  
String substring(int beginIndex, int endIndex)  
'''
```

****Simple Example:****

```
'''java  
  
String str = "Hello World";  
  
String sub1 = str.substring(6); // "World"  
  
String sub2 = str.substring(0, 5); // "Hello"  
'''
```

****Complex Example:****

```
'''java  
  
String url = "https://www.example.com/products/item123";  
  
// Extract domain  
  
int start = url.indexOf("//") + 2;  
  
int end = url.indexOf("/", start);  
  
String domain = url.substring(start, end); // "www.example.com"  
  
  
// Extract item ID  
  
String itemId = url.substring(url.lastIndexOf('/') + 1); // "item123"
```

```
'''
```

```
#### f. lastIndexOf()
```

```
**Syntax:**
```

```
```java
```

```
int lastIndexOf(int ch)
```

```
int lastIndexOf(int ch, int fromIndex)
```

```
int lastIndexOf(String str)
```

```
int lastIndexOf(String str, int fromIndex)
```

```
'''
```

```
Simple Example:
```

```
```java
```

```
String str = "Hello World";
```

```
int lastL = str.lastIndexOf('l'); // 9
```

```
int lastLo = str.lastIndexOf("lo"); // 3
```

```
'''
```

```
**Complex Example:**
```

```
```java
```

```
String path = "/usr/local/bin/java/home/bin/python";
```

```
// Find the last bin directory
```

```
int lastBinPos = path.lastIndexOf("/bin/");
```

```
if (lastBinPos != -1) {
```

```
 int nextSlash = path.indexOf("/", lastBinPos + 5);
```

```
 String lastBinDir = nextSlash == -1 ?
```

```
 path.substring(lastBinPos + 5) :
```

```
 path.substring(lastBinPos + 5, nextSlash);
```

```
 System.out.println(lastBinDir); // "python"
 }
 ...
```

```
g. charAt()
```

```
Syntax:
```

```
```java
char charAt(int index)
...`
```

```
**Simple Example:**
```

```
```java
String str = "Java";
char first = str.charAt(0); // 'J'
...`
```

```
Complex Example:
```

```
```java
String password = "S3cr3tP@ss";
boolean hasUpper = false, hasLower = false, hasDigit = false, hasSpecial = false;

for (int i = 0; i < password.length(); i++) {
    char c = password.charAt(i);
    if (Character.isUpperCase(c)) hasUpper = true;
    else if (Character.isLowerCase(c)) hasLower = true;
    else if (Character.isDigit(c)) hasDigit = true;
    else hasSpecial = true;
}
```

```
System.out.println("Password meets complexity: " +  
    (hasUpper && hasLower && hasDigit && hasSpecial));  
'''
```

h. equals() and equalsIgnoreCase()

****Syntax:****

```
```java  
boolean equals(Object anObject)
boolean equalsIgnoreCase(String anotherString)
'''
```

**\*\*Simple Example:\*\***

```
```java  
String str1 = "Hello";  
String str2 = "hello";  
boolean exactMatch = str1.equals(str2); // false  
boolean ignoreCaseMatch = str1.equalsIgnoreCase(str2); // true  
'''
```

****Complex Example:****

```
```java  
String[] validUsers = {"admin", "superuser", "moderator"};
String inputUser = "ADmin";

boolean isAuthorized = false;
for (String user : validUsers) {
 if (user.equalsIgnoreCase(inputUser)) {
```

```
 isAuthorized = true;
 break;
 }
}

System.out.println("User authorized: " + isAuthorized); // true
'''
```

### i. compareTo()

**\*\*Syntax:\*\***

```
'''java
int compareTo(String anotherString)
'''
```

**\*\*Simple Example:\*\***

```
'''java
String a = "apple";
String b = "banana";
int result = a.compareTo(b); // negative (a comes before b)
'''
```

**\*\*Complex Example:\*\***

```
'''java
String[] names = {"John", "Alice", "Bob", "Zoe", "Charlie"};
Arrays.sort(names); // Uses compareTo internally
```

```
// Custom sorting by length then alphabetically
```

```
Arrays.sort(names, (s1, s2) -> {
```

```

 int lengthCompare = Integer.compare(s1.length(), s2.length());

 return lengthCompare != 0 ? lengthCompare : s1.compareTo(s2);
});

// Result: ["Bob", "John", "Zoe", "Alice", "Charlie"]
'''

```

## ## 2. StringBuffer Methods

### ### a. insert()

**\*\*Syntax:\*\***

```

'''java

StringBuffer insert(int offset, [various types])
'''

```

**\*\*Simple Example:\*\***

```

'''java

StringBuffer sb = new StringBuffer("Hello ");

sb.insert(6, "World"); // "Hello World"
'''

```

**\*\*Complex Example:\*\***

```

'''java

StringBuffer template = new StringBuffer("Dear [name], your balance is [amount].");

Map<String, String> values = new HashMap<>();

values.put("name", "John Doe");

values.put("amount", "$1,234.56");

int nameStart = template.indexOf("[name]");

```

```
template.replace(nameStart, nameStart + 6, values.get("name"));
```

```
int amountStart = template.indexOf("[amount]");
```

```
template.replace(amountStart, amountStart + 8, values.get("amount"));
```

```
// Result: "Dear John Doe, your balance is $1,234.56."
```

```
'''
```

```
b. deleteCharAt() and delete()
```

```
Syntax:
```

```
```java
```

```
StringBuffer deleteCharAt(int index)
```

```
StringBuffer delete(int start, int end)
```

```
'''
```

```
**Simple Example:**
```

```
```java
```

```
StringBuffer sb = new StringBuffer("Hello World");
```

```
sb.deleteCharAt(5); // "HelloWorld"
```

```
sb.delete(5, 10); // "Hello"
```

```
'''
```

```
Complex Example:
```

```
```java
```

```
StringBuffer html = new StringBuffer("<div><p>Some <b>bold</b> text</p></div>");
```

```
// Remove all HTML tags
```

```
while (true) {
```

```
    int openTag = html.indexOf("<");
```

```

        if (openTag == -1) break;

        int closeTag = html.indexOf(">", openTag);

        if (closeTag == -1) break;

        html.delete(openTag, closeTag + 1);
    }

    // Result: "Some bold text"
    ...

```

These examples cover all the requested methods with both simple and complex implementations, demonstrating various parameters and use cases.

Java String and StringBuffer Methods Explained with Examples

Here's a detailed explanation of the requested methods with simple and complex examples for each:

String Methods

1. indexOf() and lastIndexOf()

****Syntax:****

```
```java
```

```
int indexOf(int ch)
```

```
int indexOf(int ch, int fromIndex)
```

```
int indexOf(String str)
```

```
int indexOf(String str, int fromIndex)
```

```
int lastIndexOf(int ch)
```

```
int lastIndexOf(int ch, int fromIndex)
```

```
int lastIndexOf(String str)
```

```
int lastIndexOf(String str, int fromIndex)
```

```
...
```

```
Simple Examples:
```

```
```java
```

```
String str = "Hello World";
```

```
// indexOf examples
```

```
int firstL = str.indexOf('l');    // 2
```

```
int secondL = str.indexOf('l', 3); // 3
```

```
int worldPos = str.indexOf("World"); // 6
```

```
// lastIndexOf examples
```

```
int lastL = str.lastIndexOf('l'); // 9
```

```
int lastLBefore8 = str.lastIndexOf('l', 8); // 3
```

```
int lastOr = str.lastIndexOf("or"); // 7
```

```
...
```

```
**Complex Example:**
```

```
```java
```

```
String log = "ERROR: File not found; WARNING: Low memory; ERROR: Timeout
occurred";
```

```
// Find all ERROR messages
```

```
int errorIndex = -1;
```

```
while ((errorIndex = log.indexOf("ERROR:", errorIndex + 1)) != -1) {
```

```
 int end = log.indexOf(';', errorIndex);
```

```
 if (end == -1) end = log.length();
```

```
 String errorMsg = log.substring(errorIndex, end).trim();
```

```
 System.out.println(errorMsg);
 }
```

```
// Output:
```

```
// ERROR: File not found
```

```
// ERROR: Timeout occurred
```

```
'''
```

```
2. substring()
```

```
Syntax:
```

```
```java
```

```
String substring(int beginIndex)
```

```
String substring(int beginIndex, int endIndex)
```

```
'''
```

```
**Simple Example:**
```

```
```java
```

```
String str = "Programming";
```

```
String sub1 = str.substring(3); // "gramming"
```

```
String sub2 = str.substring(3, 7); // "gram"
```

```
'''
```

```
Complex Example:
```

```
```java
```

```
String email = "john.doe@example.com";
```

```
// Extract username and domain
```

```
int atPos = email.indexOf('@');
```

```
if (atPos != -1) {
```

```

String username = email.substring(0, atPos);    // "john.doe"
String domain = email.substring(atPos + 1);    // "example.com"

// Extract top-level domain
int lastDot = domain.lastIndexOf('.');
if (lastDot != -1) {
    String tld = domain.substring(lastDot + 1);    // "com"
}
}
'''

```

3. charAt()

****Syntax:****

```

'''java
char charAt(int index)
'''

```

****Simple Example:****

```

'''java
String str = "Java";
char first = str.charAt(0); // 'J'
char last = str.charAt(str.length()-1); // 'a'
'''

```

****Complex Example:****

```

'''java
String password = "P@ssw0rd";
boolean hasUpper = false, hasLower = false, hasDigit = false, hasSpecial = false;

```

```

for (int i = 0; i < password.length(); i++) {
    char c = password.charAt(i);
    if (Character.isUpperCase(c)) hasUpper = true;
    else if (Character.isLowerCase(c)) hasLower = true;
    else if (Character.isDigit(c)) hasDigit = true;
    else if (!Character.isLetterOrDigit(c)) hasSpecial = true;
}

System.out.println("Password meets complexity: " +
    (hasUpper && hasLower && hasDigit && hasSpecial));
'''

```

4. valueOf()

****Syntax:****

```

'''java
static String valueOf(primitive or Object)
'''

```

****Simple Example:****

```

'''java
String numStr = String.valueOf(123);    // "123"
String boolStr = String.valueOf(true);  // "true"
String objStr = String.valueOf(new Object()); // Object's toString()
'''

```

****Complex Example:****

```

'''java

```

```
Object[] mixedArray = {42, 3.14, "text", true, null};
```

```
StringBuilder result = new StringBuilder();
```

```
for (Object obj : mixedArray) {
```

```
    result.append(String.valueOf(obj)) // Handles null safely
```

```
        .append(" | ");
```

```
}
```

```
System.out.println(result.toString());
```

```
// Output: "42 | 3.14 | text | true | null | "
```

```
'''
```

```
## StringBuffer Methods
```

```
### 1. setCharAt()
```

```
**Syntax:**
```

```
```java
```

```
void setCharAt(int index, char ch)
```

```
'''
```

```
Simple Example:
```

```
```java
```

```
StringBuffer sb = new StringBuffer("Hello");
```

```
sb.setCharAt(1, 'a'); // "Hallo"
```

```
'''
```

```
**Complex Example:**
```

```
```java
```

```
StringBuffer encrypted = new StringBuffer("Khoor#Zrug#123");
```

```
// Simple Caesar cipher decryption (shift back by 3)
```

```
for (int i = 0; i < encrypted.length(); i++) {
```

```
 char c = encrypted.charAt(i);
```

```
 if (Character.isLetter(c)) {
```

```
 char decrypted = (char)(c - 3);
```

```
 encrypted.setCharAt(i, decrypted);
```

```
 }
```

```
}
```

```
System.out.println(encrypted.toString()); // "Hello World 123"
```

```
'''
```

```
2. replace()
```

```
Syntax:
```

```
```java
```

```
StringBuffer replace(int start, int end, String str)
```

```
'''
```

```
**Simple Example:**
```

```
```java
```

```
StringBuffer sb = new StringBuffer("I like apples");
```

```
sb.replace(7, 13, "oranges"); // "I like oranges"
```

```
'''
```

```
Complex Example:
```

```
```java
```

```

StringBuffer template = new StringBuffer(
    "Dear [customer], your order [orderid] is ready for pickup.");

Map<String, String> values = new HashMap<>();
values.put("customer", "John Smith");
values.put("orderid", "ORD-2023-456");

int start;
while ((start = template.indexOf("[") != -1) {
    int end = template.indexOf("]", start);
    if (end == -1) break;

    String key = template.substring(start + 1, end);
    String replacement = values.getOrDefault(key, "[UNKNOWN]");
    template.replace(start, end + 1, replacement);
}

System.out.println(template.toString());
// Output: "Dear John Smith, your order ORD-2023-456 is ready for pickup."
'''

```

3. insert()

****Syntax:****

```
```java
```

```
StringBuffer insert(int offset, [various types])
```

```
```
```

****Simple Example:****

```
```java
```

```
StringBuffer sb = new StringBuffer("Hello World");
```

```
sb.insert(5, ","); // "Hello, World"
```

```
```
```

****Complex Example:****

```
```java
```

```
StringBuffer csvData = new StringBuffer("John,Doe,35,New York");
```

```
// Format as a table row
```

```
int pos = 0;
```

```
int fieldCount = 1;
```

```
while ((pos = csvData.indexOf(",", pos)) != -1) {
```

```
 String insertText = "</td><td>";
```

```
 csvData.insert(pos, insertText);
```

```
 pos += insertText.length() + 1;
```

```
 fieldCount++;
```

```
}
```

```
// Add HTML tags
```

```
csvData.insert(0, "<tr><td>");
```

```
csvData.append("</td></tr>");
```

```
System.out.println(csvData.toString());
```

```
// Output: "<tr><td>John</td><td>Doe</td><td>35</td><td>New York</td></tr>"
```

```
```
```

4. reverse()

****Syntax:****

```
```java
```

```
StringBuffer reverse()
```

```
```
```

****Simple Example:****

```
```java
```

```
StringBuffer sb = new StringBuffer("Hello");
```

```
sb.reverse(); // "olleH"
```

```
```
```

****Complex Example:****

```
```java
```

```
StringBuffer palindromeChecker = new StringBuffer("racecar");
```

```
StringBuffer original = new StringBuffer(palindromeChecker.toString());
```

```
if (palindromeChecker.reverse().toString().equals(original.toString())) {
```

```
 System.out.println(original + " is a palindrome!");
```

```
} else {
```

```
 System.out.println(original + " is not a palindrome.");
```

```
}
```

```
// Output: "racecar is a palindrome!"
```

```
```
```

5. delete() and deleteCharAt()

****Syntax:****

```
```java
```

```
StringBuffer delete(int start, int end)
```

```
StringBuffer deleteCharAt(int index)
```

```
'''
```

```
Simple Examples:
```

```
```java
```

```
StringBuffer sb1 = new StringBuffer("Hello World");
```

```
sb1.delete(5, 11); // "Hello"
```

```
StringBuffer sb2 = new StringBuffer("Programming");
```

```
sb2.deleteCharAt(2); // "Proramming"
```

```
'''
```

```
**Complex Example:**
```

```
```java
```

```
StringBuffer html = new StringBuffer(
```

```
 "<div><p>Some bold text <!-- comment --></p></div>");
```

```
// Remove all HTML tags and comments
```

```
while (true) {
```

```
 int openTag = html.indexOf("<");
```

```
 if (openTag == -1) break;
```

```
 int closeTag = html.indexOf(">", openTag);
```

```
 if (closeTag == -1) break;
```

```
// Check if it's a comment
```

```
if (html.substring(openTag, Math.min(openTag+4, html.length())).equals("<!--")) {
```

```
 int commentEnd = html.indexOf("-->", openTag);
```

```
 if (commentEnd != -1) {
```

```
 html.delete(openTag, commentEnd + 3);
 } else {
 html.delete(openTag, closeTag + 1);
 }
} else {
 html.delete(openTag, closeTag + 1);
}
}

System.out.println(html.toString().trim()); // "Some bold text"
'''
```

These examples demonstrate the versatility of String and StringBuffer methods in Java, showing both basic usage and more advanced scenarios. The StringBuffer methods are particularly useful when you need to perform multiple modifications to a string efficiently.